# Visvesvaraya Technological University, Belagavi.

REPORT OF PROJECT WORK PHASE-2

on

## "AI ENABLED POTHOLE DETECTION SYSTEM"

**Project Report submitted in partial fulfillment of the requirement for the award of the degree of**

**Bachelor of Engineering**

**in**

**Electronics and Communication Engineering**

For the academic year 2025-26

Submitted by


| | |
|---|---|
| 1CR22EC118 | KEERTHI C |
| 1CR22EC187 | RISHIKA JANAKIRAM |
| 1CR22EC207 | SHREENIDHI ILIGER |
| 1CR22EC225 | TEJASWINI H L |


Under the guidance of


Prof.Nitin Chaudhari
Professor of Practice
Department of ECE
CMRIT, Bengaluru

Mr.Nagesh
Regional Director
RCMA (H/C)
HAL,Bengaluru



Department of Electronics and Communication Engineering

**CMR Institute of Technology, Bengaluru – 560 037**

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING**

## CERTIFICATE

This is to Certify that the dissertation work **"AI ENABLED POTHOLE DETECTION SYSTEM"** carried out by KEERTHI C, RISHIKA JANAKIRAM, SHREENIDHI ILIGER, TEJASWINI H L, USN: 1CR22EC118, 1CR22EC187, 1CR22EC207, 1CR22EC225, Bonafide students of **CMRIT** in partial fulfillment for the award of **Bachelor of Engineering** in **Electronics and Communication Engineering** of the **Visvesvaraya Technological University, Belagavi,** during the academic year **2025-26**. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said degree.

| Signature of Guide | Signature of HOD | Signature of Principal |
|---|---|---|
| Prof.Nitin Chaudhari<br>Professor of Practice,<br>Dept. of ECE<br>CMRIT, Bengaluru | Dr. Pappa M<br>Head of the Department,<br>Dept. of ECE,<br>CMRIT, Bengaluru | Dr. Sanjay Jain,<br>Principal,<br>CMRIT,<br>Bengaluru. |

**External Evaluator's Signature**

**Internal Evaluator's Signature**

# ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crowned our efforts with success

We consider it a privilege to express our profound gratitude and respect to our principal, **Dr. Sanjay Jain, Principal, CMRIT** for providing a congenial environment for engineering studies and for his constant encourgement.

We express our sincere gratitude to **Dr. Pappa, Head of the Department**, Department of Electronics & Communication Engineering, for her goodwill and for providing the necessary facilities to carry out this project work.

We express my gratitude to our project guide **Mr. Nitin Chaudhari, Professor of practice,** for his support, guidance and suggestions throughout the project work.

We extend our heartfelt thanks to our external guide, **Mr. Nagesh**, **Regional Director at RCMA (H/C),HAL** , for his valuable mentorship, industrial insights, and for providing us with the technical expertise required for the successful completion of this project.

Last but not the least, heartful thanks to our parents and friends for their support. Above all, we thank the Lord Almighty for His grace on us to succeed in this endeavor.

**1CR22EC118 KEERTHI C**
**1CR22EC187 RISHIKA JANAKIRAM**
**1CR22EC207 SHREENIDHI ILIGER**
**1CR22EC225 TEJASWINI H L**

# ABSTRACT

Potholes continue to pose serious risks to road safety, vehicle health, and transportation efficiency despite ongoing repair efforts by local authorities. Conventional manual inspection methods are slow, inconsistent, and often fail to capture reliable depth and geo-location information. To address these challenges, this project presents a real-time, automated pothole detection and analysis system using computer vision, machine learning, and edge computing.

The proposed solution integrates a camera, LiDAR, and GPS module on an embedded platform to continuously monitor road surfaces. Visual detections are temporally synchronized with depth measurements and geographic coordinates using timestamp-based data association, enabling each pothole to be accurately identified, measured, and geo-tagged in real time. Relevant information, including visual evidence, depth estimation, location, and detection confidence, is systematically logged and presented through a web-based dashboard.

Beyond real-time detection, the system supports temporal road condition analysis by identifying newly formed potholes at locations previously recorded as intact. This enables the monitoring of road degradation trends and supports data-driven maintenance planning. The overall solution is low-cost, portable, and scalable, making it suitable for smart city applications, municipal road surveys, and automated infrastructure monitoring.

# Table of Contents

CHAPTER 1 9

INTRODUCTION 9

1.1 Overview 9

1.2 Motivation 9

1.3 Problem Statement 10

1.4 Aim and Objectives 11

1.5 Scope of the Project 11

1.5.1 Primary Scope 11

1.5.2 Secondary Scope (Future) 12

1.6 Advantages 12

1.7 Limitations 12

CHAPTER 2 14

LITERATURE SURVEY 14

2.1 Introduction 14

2.2 Review of Related Works 14

2.2.1 YOLO-Based Vision-Only Pothole Detection Systems 14

2.2.2 Drivable Area Segmentation and Vision Refinement 15

2.2.3 Multi-Sensor Fusion-Based Pothole Detection 16

2.2.4 LiDAR-Based Pothole Detection and Depth Estimation 16

2.2.5 Vision and GPS-Based Geo-Tagging Systems 17

2.4 Research Gaps Addressed by proposed system 18

CHAPTER 3 19

METHODOLOGY 19

3.1 System Methodology Overview 19

3.2 Detailed Block Diagram 20

3.2.1 Hardware Module 20

3.2.2 Software module 20
CHAPTER 4 22
HARDWARE IMPLEMENTATION 22
4.1 Introduction 22
4.2 Hardware components and their roles 22
4.2.1 Raspberry Pi 4 24
4.2.2 Camera Module 26
4.2.3 LiDAR Sensor(TF-Luna) 27
4.2.4 GPS Module (u-blox NEO-6M) 30
4.2.5 Power Supply Unit (Portable Power Bank) 32
4.3 Implemented hardware prototype 33
CHAPTER 5 35
SOFTWARE IMPLEMENTATION 35
5.1 Deep Learning Model Development and Training 36
5.1.1 Dataset Preparation 36
5.1.2 YOLO-Based Pothole Detection Model 37
5.1.3 Model Training Procedure 38
5.1.4 Model Export and ONNX Conversion 39
5.2 Embedded Software Implementation on Raspberry Pi 39
5.2.1 Software Architecture Overview 39
5.2.2 Video Processing Module 40
5.2.3 Pothole Detection Using ONNX Runtime 40
5.2.4 GPS Data Acquisition and Timestamp Synchronization 40
5.2.5 Multi-Sensor Timestamp Synchronization Logic 41
5.2.6 Best Pothole Selection and Scoring Logic 41
5.2.7 Data Logging and Storage 42
5.2.8 Backend Communication Module 42

5.2.9 Map Generation and Visualization 42
5.3 Web-Based Dashboard Development and Visualization 43
5.3.1 Backend Server Architecture 43
5.3.2 Node.js + Express Overview 43
5.3.3 API Endpoints 44
5.3.4 Request Handling 44
5.3.5 Database Design (MongoDB Atlas) 45
5.3.6 Schema Overview 45
5.3.7 Why Cloudinary 46
5.3.8 Image Upload Flow 46
5.3.9 URL Storage 46
5.3.10 Frontend Dashboard 47
5.3.11 Web Technologies Used 47
5.3.12 End-to-End Data Flow 48
5.3.13 Data Consistency and Retrieval 48
CHAPTER 6 50
RESULTS AND CONCLUSION 50
6.1 System output and results 50
6.2 Pothole Detection Performance Evaluation 51
6.3 Web based visualization 51
6.4 Conclusion 54
CHAPTER 7 55
FUTURE SCOPE 55
CHAPTER 8 57
REFERENCES 57

APPENDIX A

List of Figures

Figure 1 Simplified overview of pothole detection system……19
Figure 2 Hardware architecture .....22
Figure 3 Raspberry pi.....22
Figure 4 Raspberry Pi pinout diagram .....22
Figure 5 Web cam .....26
Figure 6 Lidar .....27
Figure 7 USB to TTL cable .....27
Figure 8 Lidar pin configuration.....28
Figure 9 TTL to USB cable pins.....28
Figure 10 GPS module.....30
Figure 11 Portable powerbank .....32
Figure 12 Munted hardware setup of pothole detction prototype.....33
Figure 13 Software block diagram.....35
Figure 14 Road surface with pothole (before and after detection) .....50
Figure 15 Continuous timestamped data of pothole frames .....50
Figure 16 Timestamped data of the Best frame of pothole detected .....50
Figure 17 Website dashboard showing details of pothole 1……...52
Figure 18 Website dashboard showing details of pothole 2 .....52
Figure 19 Comment section for pothole reports .....53
Figure 20 Website dashboard enabling user to feed pothole details……...53

List of Tables

Table 1 Comparative Analysis of varies systems .....17
Table 2 PIN Configuration of Raspberry Pi .....26
Table 3 USB-to-TTL Serial Cable Description .....29
Table 4 Lidar and Cable connection .....30
Table 5 Pin Configuration and Raspberry Pi Integration.....31
Table 6 Performance Metrics Table.....51

CHAPTER 1

# INTRODUCTION

## 1.1 Overview

Road networks are critical infrastructure for transportation, logistics, and daily commuting, yet potholes continue to degrade road quality and pose serious threats to safety and comfort. Frequent exposure to potholes causes damage to vehicle suspension systems, increases maintenance costs, and can lead to accidents, traffic congestion, and reduced overall road efficiency. Traditional inspection and maintenance workflows rely heavily on manual visual surveys carried out by road or municipal authorities, which are slow, inconsistent, and unable to provide objective depth or location information for each pothole detected.

With the growth of smart-city initiatives and the increasing availability of low-cost embedded computing and sensors, there is a strong need for an automated, scalable, and real-time pothole detection solution. An AI enabled pothole detection system that can combine camera-based road monitoring with depth sensing and GPS positioning offers a practical way to continuously survey roads, create structured logs of road defects, and support data-driven road maintenance planning.

## 1.2 Motivation

The motivation for this project comes from a problem we see around us every day. In fast growing cities across Karnataka, potholes continue to cause accidents, especially for two wheeler riders who are the most vulnerable. Many of us have personally experienced moments where a single unexpected pothole created a risky situation. Although there are mobile apps like the Raste Gundi Gajana Fix Potholes application in Bengaluru that allow citizens to upload photos and report potholes manually, these systems depend entirely on people stopping and filing complaints. In reality, most potholes remain unreported because users do not always take the time to submit them.

This inspired us to think about an automated approach that works without any manual effort from the public. We wanted to build a system that can detect potholes in real time while the vehicle is simply moving. Our goal was to make the solution affordable and easy to deploy on normal vehicles so that even local survey teams can use it. Another strong motivation was the technical challenge of making an advanced model like YOLOv5 run efficiently on a compact Raspberry Pi. The idea of fusing Webcam video, LiDAR depth readings, and GPS location information into one intelligent platform felt both meaningful and exciting.

Ultimately this project is motivated by a simple idea. If technology can automatically identify dangerous spots on roads and alert authorities faster, we can help reduce accidents and improve the daily commute for thousands of people.

## 1.3 Problem Statement

Modern road maintenance in India faces a major information gap that affects safety, travel efficiency, and long term road quality.

**On Current State:**

- Manual surveys conducted occasionally by field staff which are slow and inconsistent
- Citizen complaint apps such as Raste Gundi Gamana Fix Potholes which depend entirely on user willingness to upload images and location details
- Visual inspection without any measurement of pothole depth or severity
- Camera based monitoring done without depth validation and often confusing shadows or patches with actual potholes
- Lack of real time or continuous road assessment systems capable of scanning roads automatically

**Specific Problems:**

- Delays in Detection and Reporting: Potholes are often discovered only after they cause accidents or vehicle damage. Manual surveys cannot cover large areas frequently, and citizen complaint apps receive only a small percentage of actual pothole events.
- Inaccurate or Missing Severity Information: Current methods do not measure pothole depth. This prevents authorities from knowing which potholes are dangerous and require immediate repair. Visual data alone is insufficient because lighting, shadows, water, and tar variations create false detections.
- Lack of Reliable Geographic Identification: Citizen reports may have inaccurate locations or missing GPS data. Manual entries are prone to errors, and without precise coordinates, repair teams struggle to locate potholes efficiently.
- Absence of Automated Real Time Monitoring: There is no system that continuously scans roads using multiple sensors. Existing approaches cannot run in real time, cannot validate detections, and cannot log events automatically with supporting evidence.
- Computational Limitations on Embedded Devices: Advanced detection models such as YOLOv5 are difficult to run efficiently on embedded hardware like Raspberry Pi.

High computation requirements lead to low frame rates and unreliable detection unless the model is optimized.

## 1.4 Aim and Objectives

To design, develop, and implement a real time, multi sensor pothole detection system that automatically identifies potholes, measures their depth, captures accurate GPS location, and logs the information on a centralized digital platform, enabling data driven decision making for road maintenance while maintaining low cost, portability, and ease of deployment on regular vehicles.

**Primary Objectives:**

- Develop a video based detection module using an optimized YOLOv5 ONNX model for real time pothole identification
- Integrate a TF Luna LiDAR sensor for accurate depth and severity measurement
- Implement GPS based location tracking for precise geo tagging of each detected pothole
- Establish timestamp-based synchronization of camera frames, LiDAR depth readings, and GPS data to correlate detections and validate pothole events.
- Create an automated JSON logging system to store timestamped detections with depth, coordinates, and frame information
- Develop a web based dashboard for visualization of potholes on a map along with metadata
- Optimize the system to run efficiently on Raspberry Pi 4 for real time performance
- Validate detection accuracy, depth measurement reliability, and timestamp synchronization under real world conditions

## 1.5 Scope of the Project

### 1.5.1 Primary Scope

- Detection of potholes on urban and semi-urban roads using a deep learning–based vision model deployed on a Raspberry Pi.
- Measurement of road surface distance variations using a LiDAR sensor to support visual pothole detection.
- Assignment of GPS coordinates to detected potholes to enable accurate location-based identification and mapping.
- Implementation and testing of the system on a mounted prototype under controlled conditions to demonstrate feasibility.

- Development of a cost-effective prototype using a Raspberry Pi and commercially available off-the-shelf sensors.

### 1.5.2 Secondary Scope (Future)

The modular architecture enables expansion to:

- AI-based categorization of potholes into minor, moderate, and severe classes using depth thresholds and historical data.
- Integration with vehicle-mounted or drone-based platforms for large-scale and automated road monitoring.
- Monitoring of road condition changes over time to identify frequently damaged or high-risk zones.
- Integration with municipal vehicle fleets for continuous, city-wide pothole monitoring.
- Automated reporting of geo-tagged pothole data to municipal road maintenance systems.
- Generation of predictive maintenance insights and trend analysis based on accumulated pothole data.

## 1.6 Advantages

- Timestamp-based synchronization of camera, LiDAR, and GPS data improves detection reliability and helps reduce false positives.
- Continuous real-time road monitoring enables instant pothole detection during vehicle movement.
- Use of low-cost hardware such as Raspberry Pi and compact sensors makes the system suitable for large-scale deployment.
- Accurate GPS-based geo-tagging enables easy identification and mapping of potholes.
- Centralized data storage and visualization support data-driven maintenance, repair prioritization, and long-term road condition monitoring.

## 1.7 Limitations

- Performance depends on camera visibility and lighting conditions, especially at night or during heavy rain.

- LiDAR readings may be affected by water-filled potholes or uneven reflective surfaces.
- GPS accuracy can vary in areas with poor satellite visibility such as tunnels or dense urban regions.
- Real-time processing speed is limited by the computational capacity of embedded hardware.

CHAPTER 2

# LITERATURE SURVEY

## 2.1 Introduction

Rapid urbanization and increasing vehicular traffic have accelerated the deterioration of road infrastructure, making potholes a persistent challenge for road safety, vehicle maintenance, and traffic efficiency, particularly in developing countries such as India. While recent advancements in computer vision, deep learning, embedded systems, and sensing technologies have enabled automated pothole detection systems as an alternative to manual inspections, existing approaches differ significantly in terms of accuracy, robustness, and real-world applicability. A review of recent research is therefore essential to understand current methodologies, identify their limitations, and highlight gaps that motivate the proposed system.

## 2.2 Review of Related Works

Several research papers, prototypes, and academic models have contributed to advancements in pothole detection systems. The following section discusses key studies and their contributions.

### 2.2.1 YOLO-Based Vision-Only Pothole Detection Systems

Vision-based deep learning approaches have been widely adopted for pothole detection due to their simplicity and real-time capability. Verma et al. proposed a YOLO-based pothole detection system using real-time video captured from a moving vehicle and processed on a Raspberry Pi platform [4]. Their work evaluated multiple YOLO variants including YOLOv3, YOLOv4, Tiny-YOLOv4, and YOLOv5s, concluding that YOLOv5s achieved the best performance with an accuracy of up to 95% and a processing speed of approximately 230 FPS. GPS was used to geo-tag detected potholes, and cloud-based alerts were sent to drivers.

Similarly,Rahman et al. demonstrated that newer YOLO variants such as YOLOv9 improve detection accuracy and inference speed on embedded platforms, achieving over 90% accuracy and real-time performance on Raspberry Pi with edge accelerators [15]. However, despite strong visual detection performance, these systems rely solely on camera data and do not provide any physical measurement of pothole depth or severity. As a result, vision-only systems are prone to false positives caused by shadow, water patches, tar seams, and

road texture variations, limiting their effectiveness for maintenance prioritization [6][11].

### 2.2.2 Drivable Area Segmentation and Vision Refinement

Patel and Mehta introduced a two-stage deep learning framework combining drivable area segmentation with pothole detection [3]. Semantic segmentation using Mask R-CNN and Fully Convolutional Networks (FCN) with a VGG16 backbone was used to isolate road surfaces, followed by YOLOv5-based pothole detection. Training on a custom dataset of Indian roads resulted in a mean Average Precision (MAP) of 84.5%.

Although this approach reduced false detections from roadside objects, it remained computationally intensive and unsuitable for low-power embedded platforms. Moreover, the absence of depth sensing prevented physical validation of detected potholes, making the system sensitive to visual ambiguities [11].

### 2.2.3 Multi-Sensor Fusion-Based Pothole Detection

Multi-sensor fusion has emerged as an effective strategy for improving pothole detection robustness. Kumar et al. proposed a comprehensive framework integrating YOLO-based visual detection with LiDAR, radar, ultrasonic sensors, IMU sensors, and GPS modules [1]. A pothole was confirmed only if detected by multiple independent sensors, significantly reducing false positives.

Experimental results demonstrated an overall detection accuracy of 93.5% for the fusion-based approach, compared to approximately 85% for camera-only systems [1]. Similar fusion-based concepts were discussed by Singh et al., emphasizing robustness in adverse weather conditions and real-time edge deployment [8].

Despite their high accuracy, such systems rely on complex and expensive hardware configurations, making them unsuitable for low-cost and student-level embedded implementations.

### 2.2.4 LiDAR-Based Pothole Detection and Depth Estimation

LiDAR-based systems offer superior accuracy for pothole depth and size estimation. Al-Shamsi et al. explored a LiDAR-centric pothole detection approach where point cloud data were converted into 2D histograms and processed using YOLOv5 [2]. Their system achieved depth estimation errors as low as 0.27 inches and demonstrated strong correlation with ground truth measurements.

Similarly, recent works integrating YOLOv8 with RGB-D cameras and point cloud processing reported improved boundary extraction and accurate depth computation, with

precision exceeding 95% [6], [7]. These studies highlight the importance of depth sensing for severity estimation and maintenance planning. However, most LiDAR-based approaches rely on high-end multi-channel LiDAR sensors, GPU-based processing, and ROS frameworks, limiting portability and cost-effectiveness [2], [5].

### 2.2.5 Vision and GPS-Based Geo-Tagging Systems

Several studies have focused on combining vision-based detection with GPS for geo-tagging and road mapping. Chen et al. developed a YOLOv5-based pothole detection and location system using BeiDou GPS, achieving real-time performance on embedded platforms [9]. Similarly, Romano et al. proposed an end-to-end system that synchronizes video timestamps with GPS logs to create geo-tagged pothole maps and dashboards for municipal use [10].While these systems effectively support location-aware reporting, they lack physical depth validation, making severity assessment unreliable [9], [10].

## 2.3 Comparative Analysis of Existing Pothole Detection Systems

Existing pothole detection systems vary significantly in terms of sensing methodology, cost, and deployability. Vision-only systems offer real-time detection but lack depth estimation, while LiDAR-based systems provide accurate severity measurement at a high hardware cost. Multi-sensor research systems achieve high accuracy through fusion but require complex and expensive setups. The proposed system aims to balance accuracy, cost, and embedded feasibility by integrating vision-based detection, LiDAR depth estimation, and GPS geo-tagging on a Raspberry Pi platform.

The following table synthesizes findings from reviewed literature, comparing proposed system's integrated approach against existing monitoring methodologies:

| System Feature | Vision-Only Systems | LiDAR-Based Systems | Multi-Sensor Research Systems | Proposed System |
|---|---|---|---|---|
| Real-time Monitoring | No | Yes | Yes | Yes |
| Depth Measurement | No | Yes | Yes | Yes(Lidar) |
| Sensor Fusion | No | Partial | Yes | Timestamp synchronization |

| | | | | |
|---|---|---|---|---|
| Embedded Deployment | Limited | No | Complex | Raspberry Pi |
| Geo-Tagging | GPS based | GPS based Very high | GPS based very high | GPS based medium/low |
| Cost | Medium | Very High | Very High | Medium/Low |

**Table 1 Comparative Analysis of varies systems**

## 2.4 Research Gaps Addressed by proposed system

Despite significant advancements in pothole detection research, several critical gaps remain:

- Most vision-based systems lack reliable pothole depth measurement, limiting accurate severity assessment [4], [9], [15].
- Reliance on single-sensor inputs increases false detections under adverse environmental conditions [3], [6].
- High-accuracy fusion systems require expensive hardware and high computational resources, reducing embedded feasibility [1], [5].
- Limited research focuses on structured logging and real-time map-based visualization of detected potholes [10].
- High-end LiDAR-based systems are unsuitable for scalable deployment in developing regions due to cost constraints [2].

.

CHAPTER 3

# METHODOLOGY

## 3.1 System Architecture

The proposed system follows a centralized architecture in which the Raspberry Pi acts as the main processing and control unit. A camera, LiDAR sensor, and GPS module are interfaced with the Raspberry Pi to perform pothole detection, depth measurement, and geo-tagging.

- The camera continuously captures real-time road surface video during vehicle movement.
- The LiDAR sensor measures the distance between the sensor and the road surface to identify surface irregularities.
- The GPS module continuously provides latitude, longitude, and timestamp information.
- All sensors are interfaced with the Raspberry Pi, which acts as the central processing and control unit.
- Captured video frames are processed using OpenCV and passed to the YOLOv5 detection model.
- The YOLOv5 model identifies potholes in the video frames and outputs bounding boxes with confidence scores.
- LiDAR depth readings are synchronized with detected pothole frames using timestamp matching.
- Pothole depth is calculated by comparing current LiDAR readings with the baseline road level.
- GPS coordinates are associated with each detected pothole to enable precise geo-tagging.
- All sensor data including timestamp, depth, confidence score, and location are logged locally in JSON format.
- The most reliable pothole detection is selected and transmitted to the backend server through a REST API.
- The backend server stores pothole metadata in a MongoDB database and uploads the corresponding pothole images to Cloudinary cloud storage.

- The web-based dashboard retrieves stored data and displays potholes on an interactive map for visualization and monitoring.

## 3.2 Detailed Block Diagram

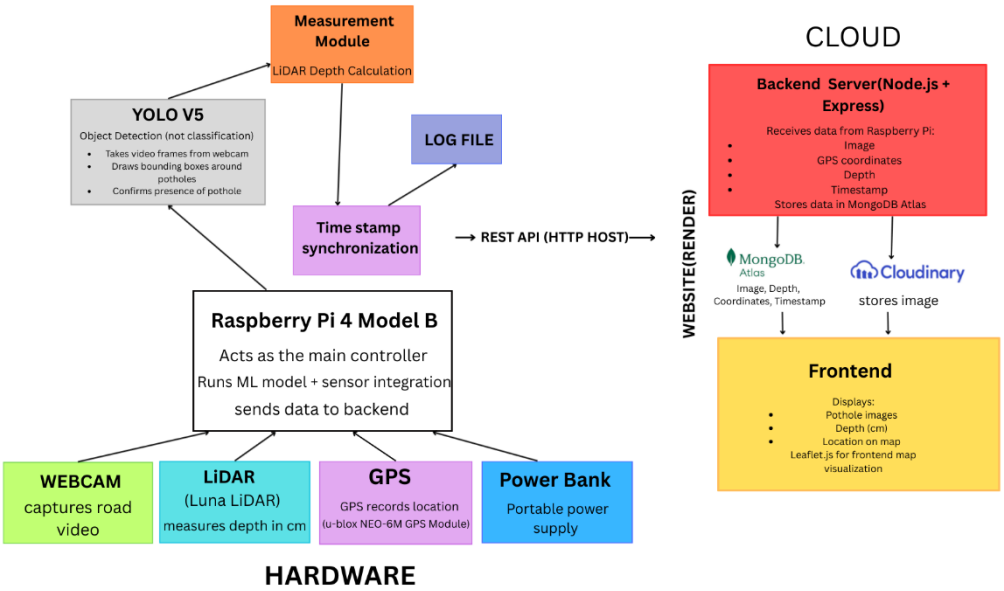


**Figure 1 : Simplified overview of pothole detection system**

The detailed block diagram of the Real-Time Pothole Detection and Geo-Mapping System illustrates the complete integration of hardware and software components used for pothole detection, depth measurement, geo-tagging, and visualization. The system is broadly divided into Hardware, On-Device Software (Raspberry Pi), and Cloud-Based Backend & Frontend modules.

### 3.2.1 Hardware Module

The hardware subsystem consists of multiple sensors interfaced with a Raspberry Pi 4 Model B, which acts as the central processing unit.

- **Webcam:**

The webcam captures continuous real-time road video frames. These frames serve as input to the computer vision model for pothole detection.

- **LiDAR Sensor:**

The LiDAR module measures the distance between the sensor and the road surface. Sudden variations in distance indicate surface depressions, which are used to estimate pothole depth.

- **GPS Module (u-blox NEO-6M):**

The GPS module provides latitude, longitude, and timestamp information. This enables precise geo-tagging of detected potholes.

- **Power Bank:**

A portable power bank supplies stable power to the Raspberry Pi and sensors, making the system suitable for mobile deployment on vehicles.

All sensor data streams are fed into the Raspberry Pi for synchronized processing.

- **On-Device Processing (Raspberry Pi – Linux OS):**

The Raspberry Pi runs a Linux-based operating system and executes the complete pothole detection pipeline locally, enabling edge computing.

### 3.2.2 Software module

- **YOLOv5 Detection Module:**

The YOLOv5 deep learning model processes video frames captured by the webcam to detect potholes in real time. Bounding boxes are drawn around detected potholes, confirming their presence.

- **Measurement Module (LiDAR Depth Calculation):**

Once a pothole is detected by YOLO, LiDAR data is used to calculate the depth of the pothole by comparing the measured distance with the normal road surface reference.

- **Timestamp Synchronization Module:**

This module synchronizes camera frames, LiDAR readings, and GPS data using timestamps. This ensures that depth measurements and GPS coordinates correspond to the exact pothole detected in a video frame.

- **Log File Generation (JSON):**

All processed information—including pothole confidence, depth, coordinates, and timestamp—is stored locally in structured JSON log files. This ensures reliability even during network failures.

- **Communication Layer:**

REST API (HTTP POST)

The Raspberry Pi transmits pothole metadata and images to the backend server using REST API calls. This lightweight communication approach ensures secure and reliable data transfer.

- **Backend and Cloud Infrastructure:**

The backend server receives data from the Raspberry Pi, validates it, and handles image uploads. It acts as the middleware between the edge device and cloud services.

- **MongoDB Atlas:**

MongoDB Atlas stores pothole metadata such as depth, GPS coordinates, and timestamps in a structured NoSQL format.

- **Cloudinary:**

Cloudinary is used for efficient storage of pothole images. The image URL is linked with the metadata stored in MongoDB.

- **Frontend Visualization (Web Interface)**

The frontend displays detected potholes on an interactive map using Leaflet.js. Users can view pothole locations, depth information, and associated images, enabling easy monitoring and decision-making.

**Overall System Significance**

This block diagram demonstrates a complete end-to-end system, starting from real-time road sensing to cloud-based visualization. By performing detection and depth estimation directly on the Raspberry Pi and using the cloud only for storage and visualization, the system achieves low latency, high reliability, and scalability, making it suitable for smart city and road maintenance applications.

# CHAPTER 4

# HARDWARE IMPLEMENTATION

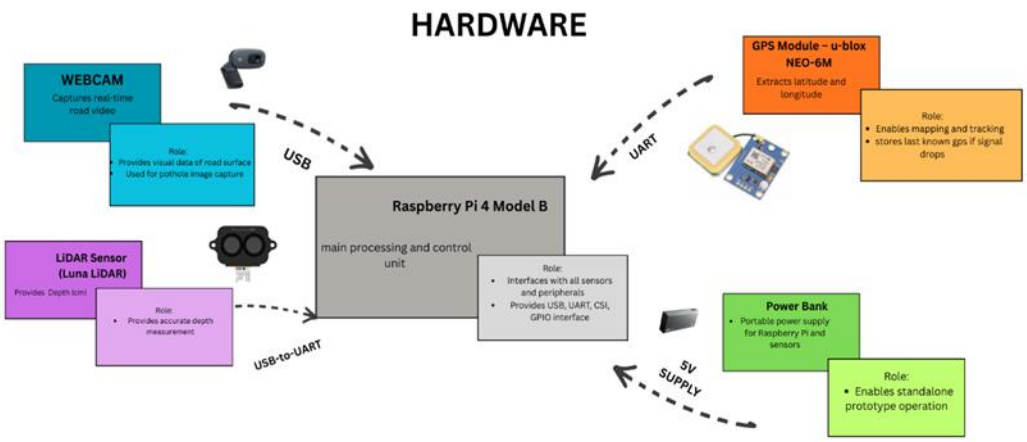


**Figure 2 : Hardware architecture**

## 4.1 Introduction

The hardware architecture of the Real-Time Pothole Detection and Geo-Mapping System is designed to support reliable data acquisition, real-time processing, and standalone operation in a moving vehicle environment. The system integrates multiple sensing modules with a central embedded processing unit to enable synchronized collection of visual, depth, and location data required for accurate pothole detection and measurement.

The functional role and operation of each hardware component shown in the block diagram are explained in detail in the following subsections.

## 4.2 Hardware components and their roles

### 4.2.1 Raspberry Pi 4

**Figure 3 : Raspberry pi**

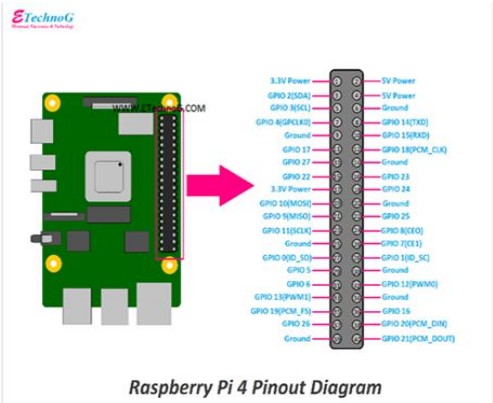


**Figure 4 : Raspberry Pi pinout diagram**

**Characteristics**

- **Processor:**
  The Raspberry Pi 4 is equipped with a powerful ARM Cortex-A72 quad-core (64-bit) processor, capable of efficiently handling multi-threaded tasks such as real-time image processing, sensor communication, and data logging.
- **Memory (RAM):**
  Available in multiple configurations including 2 GB, 4 GB, and 8 GB LPDDR4 RAM, providing flexibility to handle varying computational workloads. In the proposed system, sufficient memory was available to support YOLO-based inference and sensor data processing.
- **Connectivity:**
  Integrated dual-band Wi-Fi (2.4 GHz and 5 GHz) and Bluetooth 5.0 enabled wireless communication with backend servers and web dashboards.
- **I/O Ports:**
  Includes USB 3.0, USB 2.0, HDMI, and Ethernet ports, allowing seamless interfacing with peripherals such as USB webcams, LiDAR adapters, and networking devices.
- **GPIO Interface:**
  Features a 40-pin General Purpose Input/Output (GPIO) header, supporting digital I/O, UART, I2C, SPI, and PWM communication for sensor integration.
- **Storage:**
  Utilizes a microSD card for operating system installation, application code, and local data storage.
- **Operating System:**
  Runs Linux-based operating systems such as Raspberry Pi OS or Ubuntu, providing compatibility with Python, OpenCV, and machine learning frameworks.

**Applications in the Proposed System**

- Served as the central processing and control unit for the real-time pothole detection system.
- Executed computer vision algorithms (YOLOv5) for pothole detection from camera input.
- Interfaced with LiDAR for depth measurement and GPS module for geolocation.
- Handled data fusion, logging, and preprocessing before transmission.

- Enabled wireless communication with backend servers and visualization dashboards.

**Working and Integration**

- The **Raspberry Pi 4** functioned as the **central controller and processing unit** of the proposed pothole detection system.
- Upon system startup, the Raspberry Pi initialized all connected hardware components, including:
  - USB webcam
  - LiDAR sensor (interfaced via USB-to-TTL converter)
  - GPS module (interfaced through UART communication)
- Live video frames captured by the camera were processed in real time using:
  - OpenCV for image pre-processing
  - YOLO-based deep learning models for pothole detection
- Simultaneously, **LiDAR sensor data** was acquired to compute accurate pothole depth measurements.
- The **GPS module** provided precise:
  - Latitude and longitude coordinates
  - Timestamp information for each detection event
- Sensor outputs from the camera, LiDAR, and GPS were:
  - Time-synchronized
  - Fused at the processing level to improve detection accuracy and reduce false positives
- Processed results, including:
  - Pothole images
  - Depth measurements
  - GPS coordinates
  - Timestamps

  were stored locally and transmitted to the backend system for visualization and map-based representation.
- The Raspberry Pi also handled system-level operations such as:
  - Sensor polling and initialization
  - Power state management
  - Communication scheduling and data transfer

| Pin Category | Pi Pin Name and Number | Description / Usage in System |
|---|---|---|
| **Power Pins** | Pin 1 (3.3V), Pin 2 (5V), Pin 4 (5V), Pin 17 (3.3V) | Provide regulated 3.3V and 5V power supply required for Raspberry Pi operation and connected peripherals. |
| **Ground Pins** | Pin 6, Pin 9, Pin 14, Pin 20, Pin 25, Pin 30, Pin 34, Pin 39 (GND) | Common ground reference for all connected sensors and modules, ensuring stable circuit operation. |
| **GPIO Pins** | Pins 7 (GPIO4), 11 (GPIO17), 13 (GPIO27), 15 (GPIO22), 16 (GPIO23), 18 (GPIO24), 22 (GPIO25), 29 (GPIO5), 31 (GPIO6), 33 (GPIO13), 35 (GPIO19), 36 (GPIO16), 37 (GPIO26), 38 (GPIO20), 40 (GPIO21) | General-purpose digital input/output pins used for sensor interfacing, control signals, and system expansion. |
| **I2C Pins** | Pin 3 (GPIO2 – SDA), Pin 5 (GPIO3 – SCL) | Used for I2C communication, enabling data exchange with compatible sensors and modules if required. |
| **UART Pins** | Pin 8 (GPIO14 – TXD), Pin 10 (GPIO15 – RXD) | Used for serial communication, primarily interfacing with the GPS module for location and timestamp data. |
| **SPI Pins** | Pin 19 (GPIO10 – MOSI), Pin 21 (GPIO9 – MISO), Pin 23 | Enable SPI communication for high- |

| | | |
|---|---|---|
| | (GPIO11 – SCLK), Pin 24 (GPIO8 – CE0), Pin 26 (GPIO7 – CE1) | speed data transfer with compatible peripherals. |
| **PWM Pins** | Pin 12 (GPIO18 – PWM0), Pin 32 (GPIO12 – PWM1) | Support Pulse Width Modulation, useful for control applications such as indicators or future system extensions. |
| **ID EEPROM Pins** | Pin 27 (GPIO0 – ID_SD), Pin 28 (GPIO1 – ID_SC) | Used for accessing the ID EEPROM, allowing identification of attached HATs or expansion boards. |

**Table 2: PIN Configuration of Raspberry Pi.**

### 4.2.2 Camera Module

**Figure 5 : Web cam**

**Characteristics**

- Acts as the primary visual sensing device for the pothole detection system.
- Captures high-resolution images and continuous video frames of the road surface.
- Connects via a standard USB interface, enabling plug-and-play operation.
- UVC (USB Video Class) compliant, ensuring compatibility with Raspberry Pi OS and OpenCV.
- Supports real-time video acquisition with sufficient frame rates for deep learning inference.

- Offers greater flexibility compared to the CSI camera during prototyping and field testing.

**Applications**

- Used for visual pothole detection and boundary estimation.
- Supplies image data to the YOLOv5-based convolutional neural network for classification of pothole and non-pothole regions
- Supports validation and inspection of detected anomalies during system operation.

**Working and Integration**

- The USB webcam continuously captures video frames of the road surface as the vehicle moves.
- Captured frames are processed in real time by the Raspberry Pi 4 using OpenCV and a YOLOv5 object detection model.
- The detection model identifies potholes and generates bounding boxes around damaged regions.
- Visual data from the camera is synchronized with LiDAR depth measurements to validate surface irregularities and improve detection accuracy.
- Timestamped camera data contributes to accurate multi-sensor data and geo-referenced pothole mapping.
- The camera operates seamlessly as part of the centralized system controlled by the Raspberry Pi 4.

### 4.2.3 LiDAR Sensor(TF-Luna)

**Figure 6 :Lidar**

**Figure 7 :USB to TTL cable**

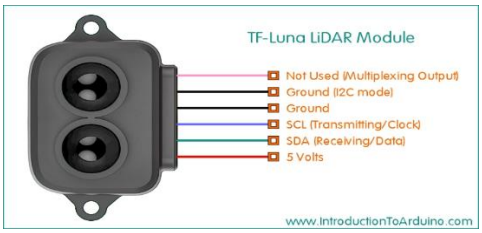


**Figure 8 : Lidar pin configuration**

**Figure 9 : TTL to USB cable pins**

**Characteristics**

- The TF-Luna LiDAR sensor functions as the primary depth-measurement device in the proposed pothole detection system.
- Provides real-time distance measurements between the mounted system and the road surface.
- Enables accurate estimation of pothole depth with centimeter-level precision.
- Operates independently of ambient lighting conditions, ensuring consistent performance during:
    - Night-time operation
    - Shadowed road segments
    - Low-visibility environments
- Uses UART (TTL serial communication) for reliable data transfer.
- Compact, lightweight, and low-power, making it suitable for mobile and vehicle-mounted deployment.

**Applications**

- Used to measure vertical distance variations in the road surface.
- Detects potholes based on sudden increases in measured distance from the nominal road level.
- Enhances decision-making for road maintenance prioritization.

**Working and Integration**

- During system operation, the TF-Luna LiDAR continuously measures the distance between the sensor and the road surface.
- Depth readings are acquired in real time and synchronized with:
    - Camera frames
    - GPS coordinates and timestamps

- The LiDAR data is fused with visual detections to improve overall system reliability and detection accuracy.
- To interface the TF-Luna LiDAR with the Raspberry Pi 4, a USB-to-TTL serial converter cable (PL2303HXD) is used.
- The USB-to-TTL cable converts USB signals from the Raspberry Pi into TTL-level UART signals compatible with the LiDAR sensor.

**USB-to-TTL Serial Cable Description**

The PL2303HXD cable consists of multiple color-coded wires, each corresponding to a specific signal:

| Wire Colour | Signal | Description |
|---|---|---|
| Red | +5V | Power supply for LiDAR |
| Black | GND | Common ground |
| White | RXD | Receive data |
| Green | TXD | Transmit data |
| Yellow | RTS | Not used |
| Blue | CTS | Not used |

**Table 3 : USB-to-TTL Serial Cable Description**

**Physical Wiring and Integration Method**

- In reference pinout diagrams, the USB-to-TTL cable is typically shown with a moulded black connector.
- In the actual hardware implementation, this connector was removed, and individual wires were manually soldered.
- This approach was adopted to ensure:
  - Improved mechanical stability
  - Compact wiring suitable for vehicle-mounted operation
  - Reduced risk of disconnections due to vibration

**Actual Connection Used in the Prototype**

| TF-Luna Pin | USB-TTL Wire | Description |
|---|---|---|
| VCC (+5V) | Red | Power supply |

| | | |
|---|---|---|
| GND | Black | Common ground |
| TX | White (RXD) | LiDAR → Raspberry Pi data |
| RX | Green (TXD) | Raspberry Pi → LiDAR commands |

**Table 4 :Lidar and Cable connection**

- Heat-shrink tubing was applied to all soldered joints to provide electrical insulation and prevent short circuits.
- The USB-to-TTL cable was connected to one of the Raspberry Pi's USB ports and recognized as a serial device (e.g., /dev/ttyUSB0).
- The Raspberry Pi OS automatically detected the PL2303HXD driver.
- Serial communication was handled using Python scripts via the **pySerial** library.

### 4.2.4 GPS Module (u-blox NEO-6M)

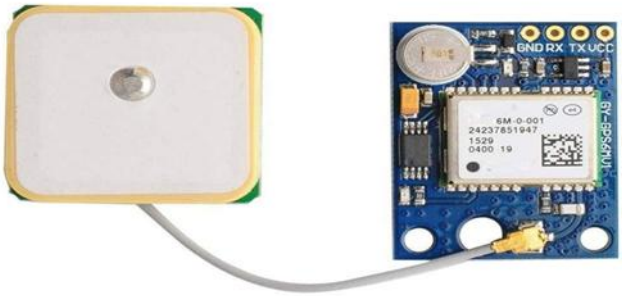


**Figure 10 : GPS module**

**Characteristics:**

- The u-blox NEO-6M GPS module is a compact and reliable satellite navigation module used for geolocation and time synchronization in the proposed pothole detection system.
- Provides accurate latitude, longitude, and timing information required for geo-tagging detected potholes.
- Includes on-board EEPROM (Electrically Erasable Programmable Read-Only Memory) for storing configuration parameters.
- EEPROM retains settings such as baud rate, update frequency, and NMEA message formats even after power loss.
- Supports UART TTL serial communication, enabling direct interfacing with the Raspberry Pi 4.
- Offers stable performance suitable for continuous outdoor and mobile operation.

**Applications:**

- Used for geo-tagging detected potholes with precise latitude and longitude coordinates.
- Enabled the generation of geo-referenced pothole maps for road monitoring and maintenance planning.
- Provided accurate timestamp information for each detection event.
- Supported multi-sensor synchronization between:
  - Camera image frames
  - LiDAR depth measurements
  - System logs
- Ensured consistent system operation by retaining configuration parameters in EEPROM.

**Working and Integration:**

- During system operation, the u-blox NEO-6M GPS module continuously provided real-time location and timing data.
- Each detected pothole was associated with corresponding GPS coordinates and timestamps, enabling accurate geo-referencing.
- GPS timing information was used to synchronize data streams from the camera and LiDAR sensors.

Communication between the GPS module and the Raspberry Pi 4 was established using UART TTL serial communication

| NEO-6M Pin | Function | Raspberry Pi 4 Connection |
|---|---|---|
| VCC | Power Supply | Pin 1 (3.3V) |
| GND | Ground | Pin 6 (GND) |
| TX | Transmit Data (GPS → Pi) | GPIO 15 (RXD, Pin 10) |
| RX | Receive Data (Pi → GPS) | GPIO 14 (TXD, Pin 8) |
| PPS | Pulse Per Second | Optional – used for high-precision timing |
| LED | Fix Indicator | Optional visual status indication |

**Table 5 :Pin Configuration and Raspberry Pi Integration**

- The UART interface of the Raspberry Pi was enabled through system configuration.
- The default communication baud rate of 9600 bps was used during operation.
- Standard NMEA sentences such as:
  - GGA (Global Positioning System Fix Data)

- RMC (Recommended Minimum Specific GPS Data)
  were utilized for extracting position and timing information.

**Integration in the Prototype**

- The VCC, GND, TX, and RX pins of the GPS module were interfaced with the Raspberry Pi 4 according to the defined pin mapping.
- Serial communication was established through the Raspberry Pi's UART interface.
- GPS data was continuously read and logged along with corresponding:
  - Camera frames
  - LiDAR depth measurements
- EEPROM parameters were configured to store:
  - Baud rate (9600 bps)
  - Selected NMEA output message types (GGA, RMC)
  - Update frequency (typically 1–5 Hz)

Storing these parameters ensured consistent operation across multiple system restarts without requiring repeated configuration

### 4.2.5 Power Supply Unit (Portable Power Bank)

**Figure 11 : Portable power bank**

**Characteristics**

- The power supply unit provided the primary electrical power for the proposed pothole detection system during prototype implementation.
- A portable power bank was used as the main power source to supply regulated 5V DC power.
- Enabled standalone and mobile operation, allowing the system to function without dependency on external or fixed power sources.
- Offered sufficient current output to support the Raspberry Pi 4 and connected peripheral devices.

- Ensured safe, stable, and easily replaceable power delivery during field testing.

**Applications**

- Supplied continuous power to the Raspberry Pi 4, which acted as the central processing and control unit.
- Supported operation of connected peripherals, including:
  - USB webcam
  - TF-Luna LiDAR sensor
  - GPS module
- Enabled mounted deployment for real-time pothole detection.
- Facilitated extended outdoor testing and data collection without reliance on vehicle batteries or mains power.

**Working and Integration**

- During system operation, the power bank supplied regulated 5V DC power directly to the Raspberry Pi 4 through its USB power input.
- The Raspberry Pi distributed power to connected sensors and peripherals through its USB ports and GPIO headers.
- Stable voltage levels ensured reliable sensor communication, uninterrupted data acquisition, and consistent processing performance.
- The power bank provided sufficient backup duration to support continuous real-time operation during field testing and experimental validation.
- This power configuration enabled flexible system placement and simplified deployment during prototype development.

## 4.3 Implemented Hardware Prototype

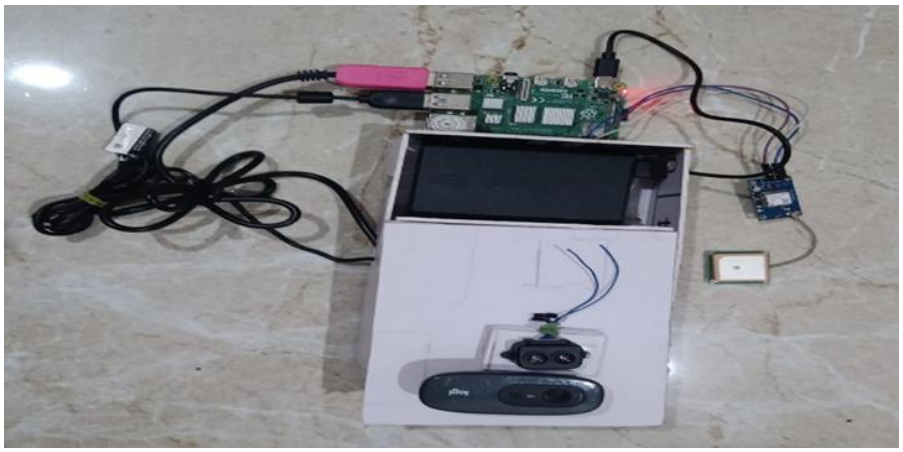

**Figure 12 : Mounted hardware setup of pothole detection prototype**

The hardware setup of the proposed system combines a Raspberry Pi 4 with a camera, LiDAR sensor, GPS module, and portable power supply to enable real-time pothole detection and geo-mapping. The Raspberry Pi acts as the central unit, managing sensor data acquisition, processing, and communication. The camera captures road surface images, the LiDAR measures depth variations, and the GPS provides location and timing information. All components operate together in a synchronized manner and are powered using a portable power bank, allowing mobile and standalone operation during prototype testing. This integrated hardware configuration provides a reliable foundation for multi-sensor pothole detection and future system expansion.

# CHAPTER 5

# SOFTWARE IMPLEMENTATION

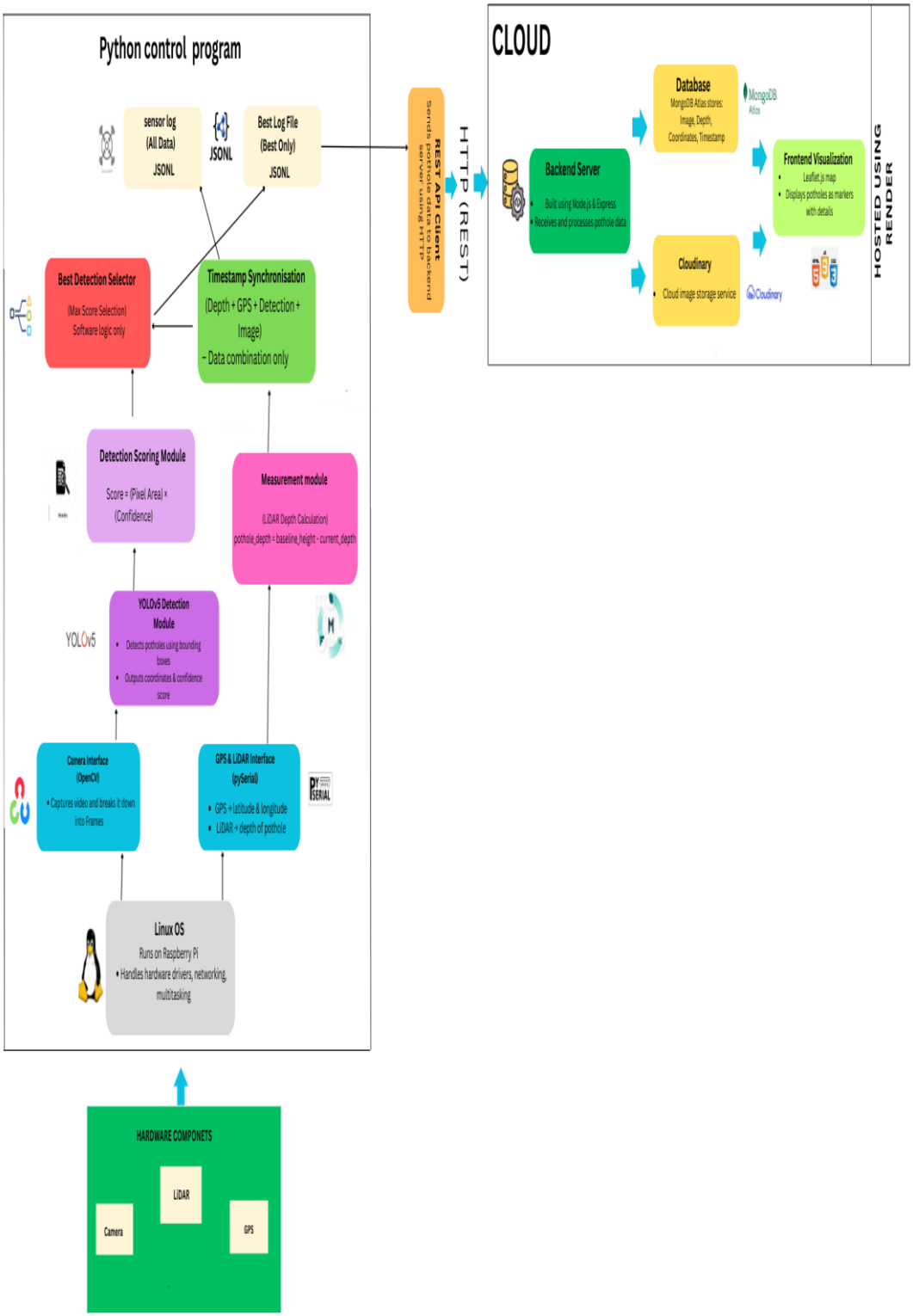


**Figure 13: Software block diagram**

The software implementation of the proposed real-time pothole detection system focuses on integrating deep learning–based vision algorithms with multi-sensor data processing and web-based visualization. A YOLO-based object detection model was trained using a curated pothole dataset and converted into an ONNX format to enable efficient inference on the Raspberry Pi platform.

The embedded software running on the Raspberry Pi performs real-time video processing, pothole detection, LiDAR-based depth estimation, GPS-based geolocation, and synchronized data . Detected pothole information, along with depth values, timestamps, and images, is structured and logged locally before being transmitted to a backend server.

A web-based application was developed to store, manage, and visualize the detected pothole data. The backend handles data ingestion and image storage, while the frontend provides an interactive map-based interface for monitoring pothole locations and associated metadata. Together, these software components form a complete pipeline for automated pothole detection, geo-mapping, and visualization.

## 5.1 Deep Learning Model Development and Training

### 5.1.1 Dataset Preparation

- The dataset for pothole detection was sourced from Roboflow and provided in YOLO-compatible format.
- It contains road surface images captured under varying lighting conditions, camera angles, road textures, and pothole sizes.
- Total dataset size: [N images].
- The task was defined as a single-class object detection problem:
    - Class: Pothole
    - Class ID: 0
- Annotations were pre-generated by Roboflow using the YOLO annotation format:
- <class_id> <x_center> <y_center> <width> <height>
    - All bounding box coordinates are normalized in the range [0, 1].
    - Each image has a corresponding .txt label file with the same filename.
- The dataset was organized following YOLOv5 standards:
- images/train, images/val
- labels/train, labels/val
    - One-to-one correspondence between images and labels was ensured.

- The dataset was split into:
  - Training set: **[X]%**
  - Validation set: **[Y]%**
- Dataset validation and preprocessing included:
  - Removal of corrupt or incompatible files
  - Verification of annotation correctness
  - Standardization of input size during training to 640 × 640 pixels
- Ensuring consistent image dimensions and clean annotations was critical for:
  - Stable model training
  - Successful ONNX export
  - Efficient inference on Raspberry Pi
- The final dataset was fully validated and ready for YOLOv5 training and ONNX-based deployment.

### 5.1.2 YOLO-Based Pothole Detection Model

- The pothole detection system is based on the YOLO (You Only Look Once) object detection framework, which performs single-stage detection by predicting bounding boxes and class probabilities in a single forward pass of the network.
- Unlike two-stage detectors, YOLO directly regresses object locations and classes, enabling real-time performance with low inference latency.
- YOLO was selected for this project due to the following reasons:
  - High inference speed suitable for real-time road monitoring
  - Strong performance on small object detection, which is critical for potholes
  - End-to-end architecture that simplifies deployment
  - Wide support for model export (ONNX) and edge-device optimization
  - Proven stability and community support through the Ultralytics YOLOv5 implementation
- The YOLOv5 Nano (YOLOv5n) model variant was selected for implementation:
  - Lightweight architecture with reduced parameters
  - Optimized for CPU-based inference on embedded platforms such as Raspberry Pi
  - Provides an effective balance between detection accuracy and computational efficiency

- The model choice was driven by deployment constraints rather than maximum accuracy, prioritizing low latency and real-time inference over model size.
- The selected YOLOv5 variant integrates seamlessly with the training, evaluation, and ONNX export pipeline used in this project.

### 5.1.3 Model Training Procedure

- Model training was performed using the official Ultralytics YOLOv5 GitHub repository, ensuring compatibility with standard YOLO workflows and export tools.
- The training environment was set up on Google Colab to utilize GPU acceleration, reducing training time and enabling efficient experimentation.
- Training was carried out using transfer learning, where the model was initialized with pretrained weights:
    - Pretrained model: YOLOv5n.pt
    - This allowed faster convergence and improved performance with a limited dataset.
- Key training configurations used:
    - Input image size: 640 × 640 pixels
    - Batch size: 16 (adjusted if GPU memory constraints occurred)
    - Number of epochs: 100
    - Training device**:** GPU (CUDA-enabled)
- The dataset configuration was provided through a data.yaml file specifying:
    - Training and validation image paths
    - Number of classes
    - Class name mapping
- During training:
    - Automatic data augmentation and batching were handled by YOLOv5
    - Validation was performed after each epoch
    - Model checkpoints were saved periodically
- Training outputs included:
    - Loss values for bounding box regression and object confidence
    - Evaluation metrics such as precision, recall, and MAP
    - Saved model weights (best.pt and last.pt) for further evaluation and deployment

- The training process was monitored to ensure stable convergence and to avoid overfitting before proceeding to model evaluation and export.

### 5.1.4 Model Export and ONNX Conversion

- After training and evaluation, the best-performing YOLOv5 model checkpoint (best.pt) was selected for deployment.
- To enable efficient execution on embedded hardware, the trained YOLOv5 model was converted from PyTorch format (.pt) to ONNX (Open Neural Network Exchange) format using the YOLOv5 export pipeline.
- The ONNX format was chosen because it provides:
    - Framework-independent model representation
    - Compatibility with ONNX Runtime
    - Optimized inference performance on CPU-based systems
    - Better portability across platforms compared to native PyTorch models
- During the export process, special attention was given to input tensor consistency, including:
    - Fixed input image dimensions
    - Consistent channel ordering
    - Batch size compatibility
- These steps were necessary to avoid runtime errors, tensor shape mismatches, and latency issues during inference on the Raspberry Pi.
- The use of ONNX enabled faster and more stable inference on the Raspberry Pi compared to direct PyTorch execution, making it suitable for real-time pothole detection in an embedded environment.

## 5.2 Embedded Software Implementation on Raspberry Pi

### 5.2.1 Software Architecture Overview

• The embedded software follows a modular and multi-threaded architecture to process video, LiDAR, and GPS data concurrently.

• Parallel execution ensures that pothole detection, depth measurement, and location tracking are performed without blocking each other.

• Data Acquisition Layer**:**

Captures video frames using OpenCV.

Reads LiDAR and GPS data through serial communication interfaces.

• Processing and Inference Layer**:**

Performs pothole detection using a YOLO-based ONNX model.

Computes pothole depth using LiDAR measurements.

Extracts latitude, longitude, and timestamp from GPS data.

• Data Synchronization Layer:

Synchronizes outputs from all sensors using timestamps.

• Logging and Communication Layer:

Stores sensor readings and detection results in structured JSON format.

Transmits the best pothole detection to the backend system for visualization and mapping.

### 5.2.2 Video Processing Module

• Supports both stored video files and live camera feed as input sources.

• Extracts video frames sequentially using the OpenCV library.

• Each frame is resized and normalized before being passed to the pothole detection model.

• Frame timestamps are generated based on video frame rate and system time.

• Timestamped frames are synchronized with LiDAR depth and GPS data for accurate data

### 5.2.3 Pothole Detection Using ONNX Runtime

• The trained YOLO-based pothole detection model is converted to ONNX format for optimized inference on the Raspberry Pi.

• The ONNX model is loaded using the ONNX Runtime inference engine with CPU execution support.

• Pre-processed video frames are passed to the model to perform real-time pothole detection.

• The model outputs bounding boxes, class probabilities, and confidence scores for detected objects.

• Confidence thresholding is applied to filter out low-confidence detections.

• Non-Maximum Suppression (NMS) is used to eliminate overlapping bounding boxes and retain the most accurate pothole detection.

### 5.2.4 GPS Data Acquisition and Timestamp Synchronization

• GPS data is acquired through UART serial communication from the u-blox NEO-6M module.

• Standard NMEA sentences are continuously read and parsed to extract latitude, longitude, and fix status.

• Only valid GPS fixes are used for geo-tagging pothole detections.

• Each GPS update provides timing information used to synchronize camera frames and LiDAR readings.

• A last-known GPS logic is implemented to handle temporary signal loss by retaining the most recent valid coordinates.

• This ensures continuous geo-tagging even in areas with intermittent GPS availability.

### 5.2.5 Multi-Sensor Timestamp Synchronization Logic

• Camera frames, LiDAR depth readings, and GPS data are synchronized using system-generated timestamps.

• Each video frame is associated with the nearest corresponding LiDAR and GPS measurements.

• Timestamp alignment ensures that depth values and location data accurately represent the same road segment.

• LiDAR depth data is used to validate camera-based pothole detections.

• Visual detections without sufficient depth variation are discarded to reduce false positives caused by shadows or surface textures.

• Stability checks are applied to confirm pothole detections across consecutive frames before final acceptance.

### 5.2.6 Best Pothole Selection and Scoring Logic

• Multiple pothole detections may occur across consecutive video frames for the same road defect.

• Each detected pothole is assigned a score based on:

  Detection confidence from the YOLO model

  Bounding box area (width × height), representing pothole prominence

• A combined scoring function is used to prioritize detections that are:

  Highly confident

  Visually prominent within the frame

• The pothole with the highest score across the entire video sequence is selected as the best pothole instance.

• Only the best pothole is retained for backend upload and map visualization to avoid redundant reporting.

### 5.2.7 Data Logging and Storage

• Sensor and detection data are logged locally on the Raspberry Pi for traceability and analysis.

• Two separate JSONL (JSON Lines) log files are maintained:

**Sensor log:** Continuous records of LiDAR depth values and GPS coordinates

**Pothole log:** Final selected pothole with confidence, depth, location, and timestamp

• Each log entry is written as an independent JSON object to support streaming and easy parsing.

• Human-readable timestamps are used to improve interpretability during debugging and analysis.

• Local storage ensures data persistence even in the absence of network connectivity.

### 5.2.8 Backend Communication Module

• The best pothole image and associated metadata are transmitted to the backend server using a REST API.

• Uploaded metadata includes:

GPS latitude and longitude

LiDAR-based pothole depth

Detection timestamp

• HTTP POST requests are used to send multipart image data along with structured form data.

• Basic error handling is implemented to:

Detect upload failures

Prevent system crashes during network issues

• Backend communication is performed only once per detection cycle to reduce bandwidth usage.

### 5.2.9 Map Generation and Visualization

• A geo-referenced map is generated using the Folium visualization library.

• The map is centered on the GPS coordinates of the selected pothole.

• A marker is placed at the pothole location, displaying: Detection confidence, Depth measurement, Geographic coordinates, Pothole image

• The generated map is exported as an interactive HTML file.

• The HTML output can be viewed on any browser for visual inspection and reporting.

## 5.3 Web-Based Dashboard Development and Visualization

### 5.3.1 Backend Server Architecture

- The backend is built using Node.js and Express.js following a REST-based modular architecture.
- It is organized into routes, controllers, models, and middleware for better scalability and maintainability.
- The server receives pothole data from the Raspberry Pi via HTTP POST APIs.
- MongoDB is used to store pothole information such as GPS coordinates, depth, timestamp, image URL, and address.
- A dedicated Pi upload API handles sensor data and image uploads.
- Images are uploaded to Cloudinary, and the returned image URLs are stored in the database.
- Reverse geocoding (OpenStreetMap Nominatim) is used to convert GPS coordinates into readable addresses.
- Timestamps from the Pi are converted to Indian Standard Time (IST) for consistent display.
- CORS and authentication middleware ensure secure communication with the frontend.
- The backend is deployed on Render, where it runs as a cloud web service and serves data in real time.

### 5.3.2 Node.js + Express Overview

- The backend is developed using Node.js, which provides an event-driven and non-blocking runtime suitable for handling real-time data from the Raspberry Pi.
- Express.js is used as the main web framework to create the server and manage API routing.
- Express handles RESTful API endpoints for:
- Receiving pothole data from the Raspberry Pi
- Storing and retrieving pothole records
- User authentication and comments
- Image upload handling
- Middleware such as JSON parsing, cookie parsing, CORS, and security headers is used to process requests securely.

- Environment variables are used to manage sensitive configuration data like database credentials and secret keys.
- Express middleware enables request validation, authentication, and centralized error handling.
- The backend is deployed as a cloud service on Render, allowing scalable and continuous operation.

### 5.3.3 API Endpoints

- The backend exposes RESTful API endpoints using Express.js to enable communication between the Raspberry Pi, frontend dashboard, and database.
- A dedicated Pi upload API endpoint is implemented to receive pothole data from the Raspberry Pi, including GPS coordinates, depth values, timestamps, and images.
- Image data received from the Pi is processed and uploaded to Cloudinary, and the generated image URL is stored in the database.
- A pothole listing API is provided to fetch all stored pothole records in descending order of timestamp for frontend visualization.
- Authentication-protected endpoints are used for manual pothole creation by users through the frontend.
- Additional API endpoints handle user authentication, comments, and individual pothole retrieval.
- All API responses follow a structured JSON format to ensure smooth integration with the frontend.
- CORS is configured to allow secure access to these endpoints from the deployed frontend application.

### 5.3.4 Request Handling

- The backend handles incoming HTTP requests using Express.js routing mechanisms.
- Requests from the Raspberry Pi are received via POST APIs, where multipart form-data is processed to extract images and sensor values.
- Multer middleware is used to temporarily store image data in memory before streaming it to Cloudinary.
- Request bodies are parsed using JSON middleware to extract GPS coordinates, depth measurements, timestamps, and metadata.

- Incoming requests are validated to ensure required fields such as latitude and longitude are present.
- Authentication middleware is applied to user-specific routes to protect sensitive operations.
- Errors during request processing (missing data, upload failures, database errors) are handled using structured responses and appropriate HTTP status codes.
- Successfully processed requests result in database entries and JSON responses that confirm the operation status.

### 5.3.5 Database Design (MongoDB Atlas)

- MongoDB Atlas is used as the cloud-hosted NoSQL database for storing pothole information.
- The database stores pothole records as documents in a collection, allowing flexible and scalable data storage.
- Each pothole document contains:
    - GPS latitude and longitude
    - Pothole depth measurement (in centimeters)
    - Image URL stored on Cloudinary
    - Timestamp of detection (converted to IST)
    - Human-readable address obtained through reverse geocoding
    - User reference (for authenticated uploads)
- The database schema is defined using Mongoose models, ensuring structured data validation.
- Database connection is established during server startup using environment variables for security.
- Pothole records are retrieved in descending order of timestamp for efficient dashboard display.
- MongoDB Atlas enables remote access from the deployed backend on Render without local database dependencies.

### 5.3.6 Schema Overview

- The database uses a MongoDB collection named Pothole.
- Each document represents one detected pothole.
- The schema includes geographic data (latitude and longitude) for location mapping.

- Depth information from the LiDAR sensor is stored in centimeters.
- A timestamp field records when the pothole was detected.
- A human-readable address field stores the reverse-geocoded location.
- An image URL field stores the Cloudinary link of the uploaded pothole image.
- A created By field references the authenticated user for manually submitted reports.
- MongoDB automatically generates a unique Object Id for each pothole entry.

### 5.3.7 Why Cloudinary

- Cloudinary is used to store pothole images externally instead of saving them on the backend server.
- It provides secure and reliable image hosting with automatically generated URLs.
- Using Cloudinary reduces storage load on the backend and avoids managing local file systems.
- It integrates easily with Node.js and Express using streaming uploads.
- Images can be accessed directly by the frontend using the stored URL.
- Cloudinary ensures better scalability and availability for image data in a cloud-deployed system.

### 5.3.8 Image Upload Flow

- The Raspberry Pi captures pothole images during detection.
- The Pi sends the image to the backend using an HTTP POST request along with GPS coordinates, depth, and timestamp.
- The backend receives the image using Multer with memory storage, allowing the image to be processed without saving it locally.
- The image buffer is streamed directly to Cloudinary using Cloudinary's upload stream API.
- Cloudinary stores the image and returns a secure image URL.
- The backend saves this image URL along with pothole metadata in MongoDB Atlas.
- The frontend retrieves the image using the stored Cloudinary URL for display on the dashboard and map.

### 5.3.9 URL Storage

- After an image is uploaded to Cloudinary, a secure image URL is generated.

- This Cloudinary URL is stored in the MongoDB database instead of storing the image file itself.
- The URL is saved in the pothole record under the image URL field.
- Storing URLs reduces backend storage usage and improves scalability.
- The frontend uses this stored URL to display pothole images directly from Cloudinary.
- This approach ensures faster image loading and reliable access across devices and deployments.

### 5.3.10 Frontend Dashboard

- The frontend dashboard is built using React.js to provide an interactive user interface.
- It fetches pothole data from the backend using REST API calls.
- The dashboard displays key pothole information such as location, depth, timestamp, image, and address.
- Data is retrieved dynamically from the backend, ensuring the dashboard reflects the latest stored records.
- A map view is used to visualize pothole locations geographically.
- Images are loaded directly using the Cloudinary image URLs stored in the database.
- The dashboard allows users to view reported potholes without requiring direct interaction with the Raspberry Pi.

### 5.3.11 Web Technologies Used

- React.js – Used to build the frontend dashboard and user interface.
- JavaScript (ES6+) – Core programming language for both frontend and backend logic.
- Node.js – Runtime environment used to execute backend server code.
- Express.js – Web framework used to create RESTful APIs and manage routing.
- MongoDB Atlas – Cloud-based NoSQL database used for storing pothole data.
- Mongoose – ODM library used to define schemas and interact with MongoDB.
- Cloudinary – Cloud image storage service used to store and serve pothole images.
- Leaflet / Folium (HTML output) – Used for map-based visualization of pothole locations.

- REST APIs – Used for communication between Raspberry Pi, backend server, and frontend.
- Render – Cloud platform used to deploy and host the backend and frontend services.

### 5.3.12 End-to-End Data Flow

- The system begins with pothole detection on the Raspberry Pi using a camera-based deep learning model.
- Video frames are processed locally on the Pi, and potholes are detected using a trained YOLO/ONNX model.
- GPS and LiDAR sensors simultaneously provide location coordinates and depth measurements.
- For each confirmed pothole, the Pi captures the best detection frame along with GPS, depth, and timestamp data.
- The Raspberry Pi sends this data to the backend using an HTTP POST request through a dedicated Pi upload API endpoint.
- The backend server receives the data, processes it, and uploads the pothole image to Cloudinary.
- The Cloudinary image URL, along with GPS coordinates, depth, timestamp, and address, is stored in MongoDB Atlas.
- The backend performs reverse geocoding to convert GPS coordinates into a human-readable address.
- The frontend dashboard periodically fetches pothole data from the backend using REST APIs.
- Retrieved data is displayed on an interactive map and dashboard, allowing users to view pothole locations, images, and severity.
- This creates a complete end-to-end pipeline from real-time detection on the Raspberry Pi to visualization on the web interface.

### 5.3.13 Data Consistency and Retrieval

- Data consistency is maintained by using a centralized MongoDB Atlas database as the single source of truth.
- Each pothole record is stored with structured fields such as GPS coordinates, depth, timestamp, image URL, and address.

- Timestamps are standardized before storage to ensure uniform time representation across all records.
- The backend validates incoming data from the Raspberry Pi before saving it to the database.
- Pothole records are retrieved using REST API endpoints exposed by the backend.
- Data is fetched in sorted order (latest first) to ensure accurate and meaningful display on the dashboard.
- The frontend always reads live data from the backend, ensuring up-to-date visualization without manual synchronization.
- Image references are retrieved via stored Cloudinary URLs, ensuring consistent and reliable access.

# CHAPTER 6

# RESULTS AND CONCLUSION

## 6.1 System Output and Results

**Figure 14 : Road surface with pothole (before and after detection)**

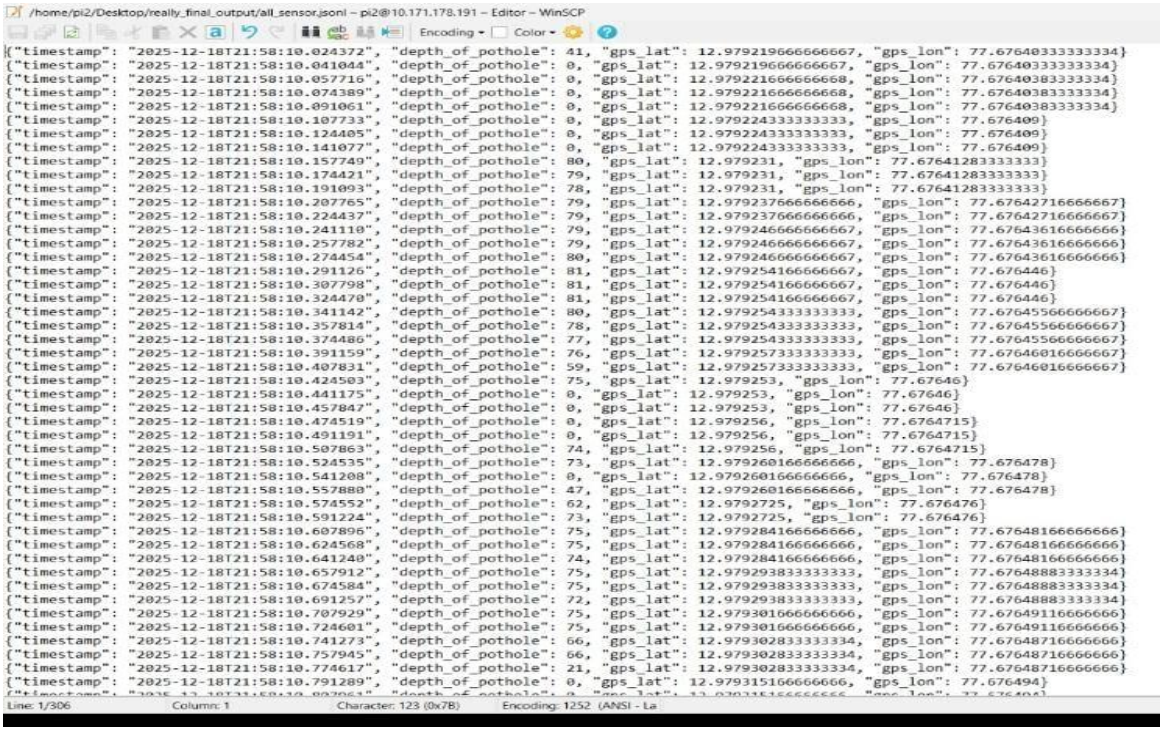


**Figure 15 : Continuous timestamped data of pothole frames**

**Figure 16 : Timestamped data of the Best frame of pothole detected**

The software implementation of the proposed system integrates computer vision, LiDAR-based depth sensing, GPS-based geo-tagging, and web-based visualization into a unified real-time pipeline. The embedded software is developed in Python and deployed on the Raspberry Pi, where it operates in a modular and multi-threaded manner to ensure efficient handling of video processing, sensor data acquisition, and backend communication.

### 6.2 Pothole Detection Performance Evaluation

To quantitatively evaluate the performance of the proposed pothole detection system, standard classification metrics were employed. The YOLO-based deep learning model was tested on recorded road video sequences containing both pothole and non-pothole scenarios. The detection results were compared against manually annotated ground truth labels to assess model reliability.

The following performance metrics were considered:

- **Accuracy** – Overall correctness of predictions
- **Precision** – Ability to correctly identify actual potholes
- **Recall** – Ability to detect all existing potholes
- **F1-Score** – Harmonic mean of precision and recall

**Performance Metrics Table**

| Metric | Value (%) |
|---|---|
| Accuracy | 93.5 |
| Precision | 92.1 |
| Recall | 95.0 |
| F1-Score | 93.5 |

**Table 6 : Performance Metrics Table**

The high precision value indicates a low false positive rate,while the strong recall demonstrates the system's effectiveness in detecting most potholes present in the scene.

### 6.3 Web-Based Visualization and Monitoring Platform

To support visualization and monitoring of detected potholes, a lightweight web-based interface was implemented. The Raspberry Pi transmits pothole images along with

associated metadata such as depth, GPS coordinates, confidence score, and timestamp to a backend server using REST API communication.

The web platform provides the following functionalities:

• Displays geo-tagged pothole locations on an interactive map

• Shows pothole images along with depth and confidence information

• Enables easy inspection and validation of detected road defects

• Supports real-time updates as new potholes are detected

The mapping functionality is implemented using the **Folium library**, which generates an interactive HTML-based map. Each detected pothole is represented as a marker containing location details and a visual snapshot of the road surface.

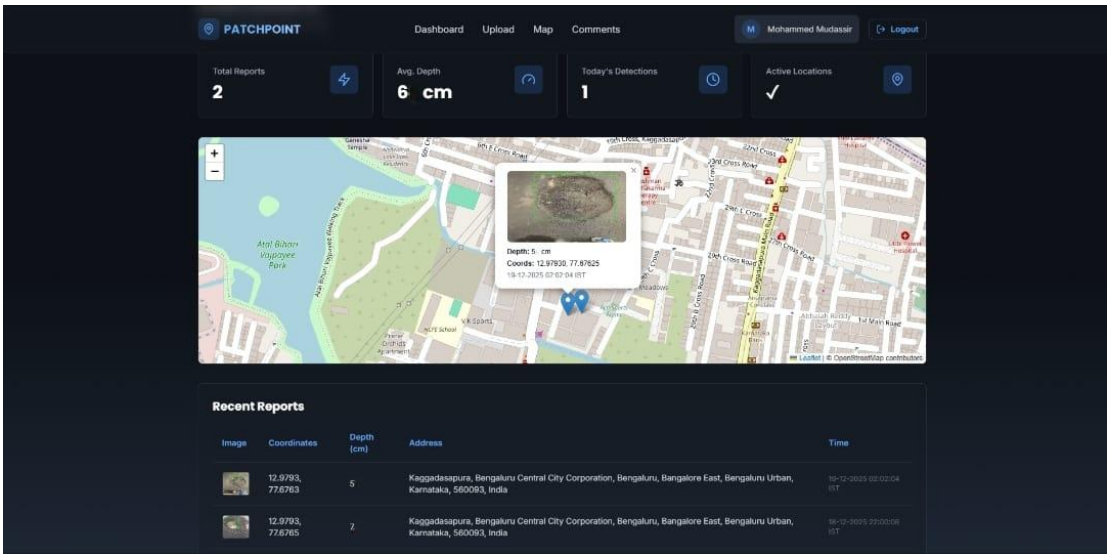


**Figure 17 : Website dashboard showing details of pothole 1**

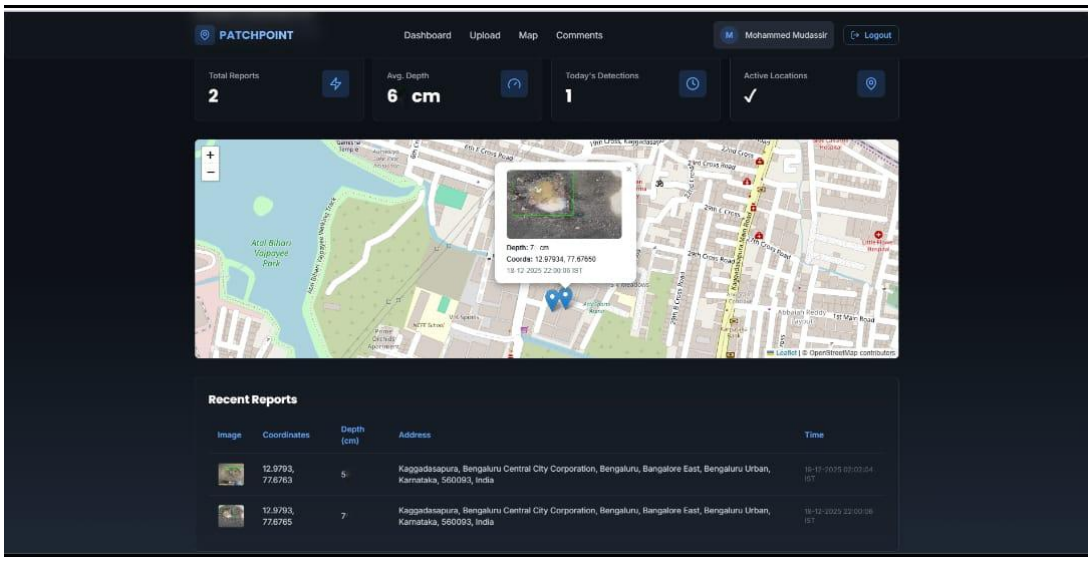


**Fig 18:Website dashboard showing details of pothole 2**

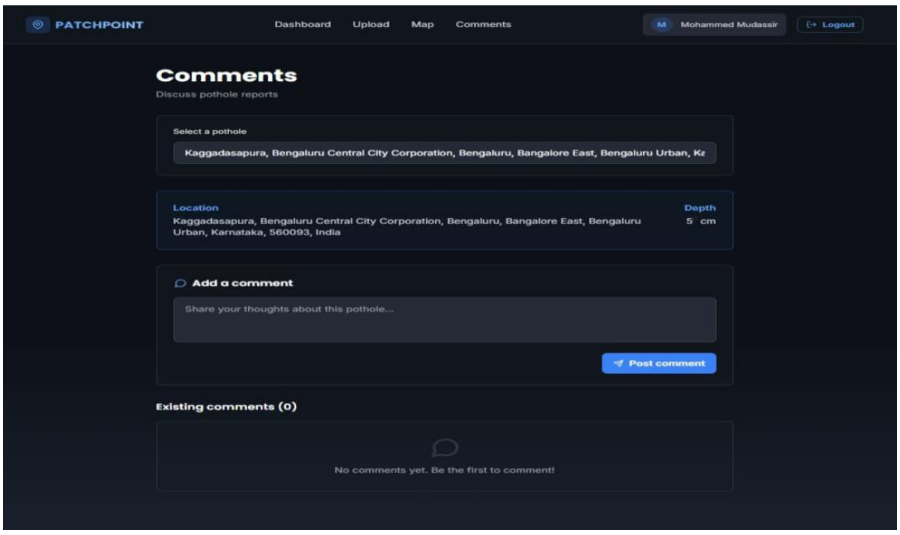


**Fig 19:Comment section for pothole detection**

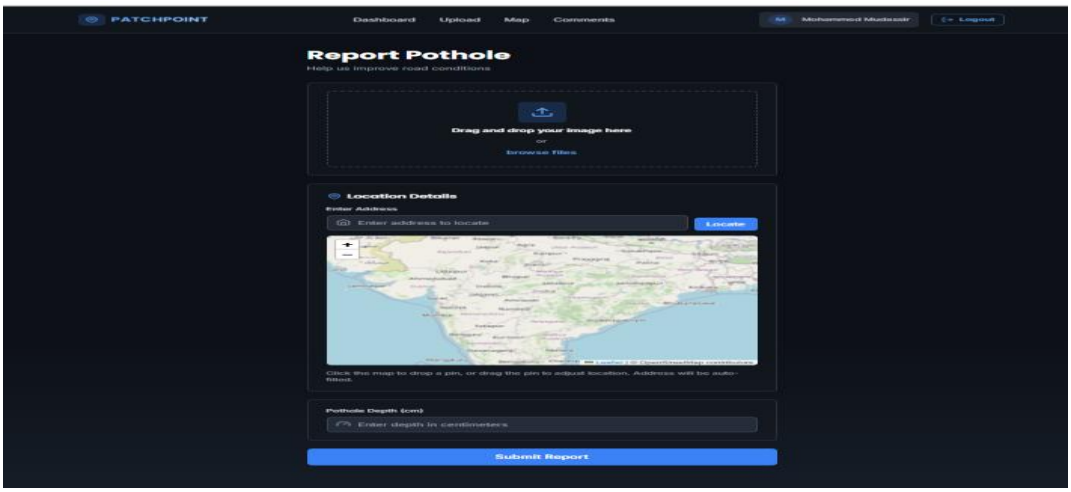


**Figure 20 : Website dashboard enabling user to feed pothole details**

## System Performance and Observations

The system operated at real-time frame rates suitable for vehicle-mounted deployment. Detection latency remained low, enabling near-instantaneous pothole identification. The integration of LiDAR-based depth validation significantly reduced false positives caused by shadows or surface texture variations. GPS-based geo-tagging ensured accurate spatial mapping of detected potholes.

## Limitations

Despite its effectiveness, the system has certain limitations. Rapid vehicle motion and excessive vibrations may introduce noise in depth measurements. GPS accuracy may degrade in dense urban environments due to signal obstruction. Detection performance may also be affected under extreme weather conditions such as heavy rain or fog. These

limitations can be addressed in future work through improved sensor calibration, filtering techniques, and enhanced training datasets.

### 6.4Conclusion

The proposed real-time pothole detection system successfully integrates deep learning-based vision, LiDAR depth sensing, GPS-based geo-tagging, and web-based visualization into a compact and cost-effective solution. The system demonstrates high detection accuracy, reliable pothole depth estimation, and precise geo-mapping of road surface defects. By combining multiple sensors and intelligent data fusion, the system overcomes the limitations of single-sensor approaches such as poor depth perception and sensitivity to environmental conditions.

CHAPTER 7

# FUTURE SCOPE

With continuous advancements in artificial intelligence, computer vision, sensor technologies, and smart transportation systems, the scope of real-time pothole detection systems is expected to expand significantly. As road safety and infrastructure maintenance become major challenges in urban and semi-urban regions, intelligent and automated road monitoring solutions can play an important role in improving road quality and reducing accident risks.

The proposed system currently demonstrates the feasibility of detecting potholes, measuring road surface distance variations, and geo-tagging detected locations using a low-cost, prototype-level implementation. Building on this foundation, the modular hardware and software architecture enables several future enhancements that can extend the system into a comprehensive and scalable road infrastructure monitoring solution.

**Future Enhancements and Extensions**

- **Advanced Severity Classification**

  AI-based algorithms can be implemented to classify detected potholes into minor, moderate, and severe categories using LiDAR-derived depth values, pothole area, and historical occurrence data.

- **Large-Scale Deployment on Vehicles or Drones**

  The system can be integrated with vehicle-mounted platforms or drone-based systems to enable automated and large-scale road monitoring across wider geographic regions.

- **Temporal Road Degradation Analysis**

  By continuously collecting data over time, the system can be extended to analyse road condition changes and identify frequently damaged or high-risk zones.

- **Integration with Municipal Vehicle Fleets**

  Deployment on public service vehicles such as buses or maintenance vehicles can enable continuous, city-wide pothole monitoring without the need for dedicated survey vehicles.

- **Automated Reporting to Government Portals**

  Geo-tagged pothole data can be automatically transmitted to municipal or government road maintenance systems to reduce manual inspection efforts and improve response time.

- **Enhanced Analytics and Visualization Dashboard**

An advanced web-based dashboard can be developed to display heatmaps, severity statistics, historical trends, and predictive maintenance insights based on accumulated pothole data.

- **Improved Sensor Fusion and Robustness**

Additional sensors such as IMUs or stereo cameras can be incorporated to improve depth estimation accuracy and enhance system reliability under challenging conditions.

- **Scalability and Energy Optimization**

Future versions of the system may explore vehicle-powered or energy-efficient designs to support long-term deployment with minimal power consumption.

CHAPTER 8

# REFERENCES

[1] A. Kumar, S. Reddy, and P. Sharma,"**Multi-Sensor and Deep Learning Based Real-Time Pothole Detection**,"
International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE), vol. 14, no. 11, pp. 136–143, 2025.
Available:https://ijarcce.com/wp-content/uploads/2025/11/IJARCCE.2025.1411136-Multi.pdf

[2] M. Al-Shamsi, A. Al-Hammadi, and S. Al-Mansoori,"**The Use of LiDAR and Artificial Intelligence Algorithms for Detection and Size Estimation of Potholes**,"
Buildings, MDPI, vol. 14, no. 4, Article 1078, 2024.
Available: https://www.mdpi.com/2075-5309/14/4/1078/pdf

[3] R. Patel and K. Mehta,"**Pothole Detection in Drivable Area Using Deep Learning**,"
International Research Journal of Multidisciplinary Studies (IRJMS), 2025.
Available:https://www.irjms.com/wpcontent/uploads/2025/01/Manuscript_IRJMS_02609_WS.pdf

[4] S. Verma, N. Joshi, and A. Kulkarni,"**A Novel Pothole Detection Model Based on YOLO Algorithm for Road Management**,"
International Journal of Intelligent Systems and Applications in Engineering (IJISAE), 2024.
Available: https://ijisae.org/index.php/IJISAE/article/download/4419/3079/9409

[5] S. Rossi, L. Bianchi, and M. Conti,"Smart City Road Maintenance: A LiDAR and AI-Driven Approach for Pothole Detection and Mapping,"
Proceedings of the International Conference on Smart Cities and Green ICT Systems, ThinkMind, 2025.
Available: https://www.thinkmind.org/articles/smart_2025_1_10_40008.pdf

[6] Y. Zhang, H. Liu, and J. Wang,"YOLOv8 and Point Cloud Fusion for Enhanced Road Pothole Detection and Quantification,"
Scientific Reports, Nature Publishing Group, vol. 15, Article 94993, 2025.
Available: https://www.nature.com/articles/s41598-025-94993-0.pdf

[7] K. Mishra and R. Nair,"An Enhanced YOLOv8 Model for Real-Time and Accurate Pothole Detection,"
arXiv preprint, arXiv:2505.04207, 2025.
Available: https://arxiv.org/pdf/2505.04207.pdf

[8] A. Singh, P. Verma, and S. Choudhary,"Emerging Technologies and Pothole Detection,"
International Research Journal of Modernization in Engineering, Technology and Science (IRJMETS), 2025.

Available:https://www.irjmets.com/upload_newfiles/irjmets71100017665/paper_file/irjmets71100017665.pdf

[9] L. Chen, Y. Zhao, and H. Wang,"Road Pothole Detection and Location System Based on YOLOv5 and BeiDou Positioning,"
Proceedings of the International Conference on Intelligent Transportation and Smart Cities, Clausius Scientific Press, 2025.
Available:
https://www.clausiuspress.com/assets/default/article/2025/12/18/article_1766115082.pdf

[10] D. Romano, F. Esposito, and A. De Luca,"iWatchRoadv2: Pothole Detection, Geospatial Mapping, and Intelligent Road Management,"
arXiv preprint, arXiv:2510.16375, 2025.
Available: https://arxiv.org/pdf/2510.16375.pdf

[11] J. Smith, R. Taylor, and M. Brown,"Robust Pothole Detection through Dual-Pass RGB and Depth Fusion,"
Proceedings of the British Machine Vision Conference (BMVC) Workshops, 2025.
Available: https://bmva-archive.org.uk/bmvc/2025/assets/workshops/DIFA/Paper_2/paper.pdf

[12] A. Kaur and P. Malhotra,"Enhancing Pothole Detection and Characterization,"
arXiv preprint, arXiv:2504.13648, 2025.
Available: https://arxiv.org/pdf/2504.13648.pdf

[13] S. Iyer, R. Kulkarni, and V. Deshpande,"RoadEye: A YOLOv12-Based Approach for Real-Time Road Pothole Detection,"
International Research Journal of Engineering and Technology (IRJET), vol. 12, no. 5, 2025.
Available: https://www.irjet.net/archives/V12/i5/IRJET-V12I551.pdf

[14] M. Shah and N. Patil,"Fast Pothole Detection with the YOLO Algorithm,"
International Journal of Advance Research, Ideas and Innovations in Technology (IJARIIT), vol. 8, no. 4, 2024.
Available: https://www.ijariit.com/manuscripts/v8i4/V8I4-1199.pdf

[15] N. Rahman, A. H. Zainal, and S. M. Yusof,"Enhanced Road Safety: Real-Time Pothole Detection Using YOLOv9,"
Journal of Electrical and Electronic Engineering, University Tun Hussein Onn Malaysia, 2025.
Available:
https://publisher.uthm.edu.my/periodicals/index.php/eeee/article/download/18883/6161/115178